

Báo Cáo Phê Bình AI: Năng Lực & Hạn Chế Của LLM Trong Kiểm Thử API

Sinh viên thực hiện: Lâm Hữu Khánh — MSSV: 23127205

Vai trò nhóm: Thành viên 1

Môn học: Kiểm thử phần mềm (Software Testing) — HW06: API Testing

Ngày lập báo cáo: 31/08/2026

Mô hình AI được đánh giá: Google Antigravity (Claude 3.7 Sonnet / Gemini 2.0 Flash)

1. Giới Thiệu & Mục đích Đánh Giá Phần Bìn

Trong khuôn khổ bài tập HW06, AI (Large Language Models) được dùng xuyên suốt quá trình thiết kế test cases, chuyển đổi từ OpenAPI, sinh test scripts Postman và xây dựng Agent Skill. Báo cáo này đưa ra đánh giá phần bìn chuyên môn (Critical Assessment) về những gì AI làm xuất sắc, những điểm mù / ảo giác (Hallucinations) nguy hiểm của AI, và lý do tại sao vai trò của K người trong vòng (Human-in-the-Loop).

2. Đánh Giá 3 Khía Cảnh Chính Của AI Trong Kiểm Thử API

graph TD

AI["**Ng Dng AI Trong API Testing**"] --> Dim1["1. Ng Cảnh Nghĩp V & Edge Cases"]

AI --> Dim2["2. Sinh Assertions & Schema Validation"]

AI --> Dim3["3. Phát Hiện Lỗi Hng Bng M (OWASP API Top 10)"]

Dim1 --> P1["+ Tc sinh Domain/BVA c nhanh"]

Dim1 --> N1["- Mù ô nhiễm trạng thái (State Pollution) & ảo giác endpoint"]

Dim2 --> P2["+ Schema JSON chu n xác cho Happy Path"]

Dim2 --> N2["- Gi nh sai HTTP Status Code & Thi u Deep Assertions"]

Dim3 --> P3["+ Nh t t các vector t n công SQLi/XSS ph bìn"]

Dim3 --> N3["- B sót hoàn toàn Broken Auth & Data Exposure trong mã ngu n"]

■ KHÍA CẢNH 1: KH NNG HIU NG CẢNH NGHĨP V & EDGE CASES

1.1 Điểm mạnh vượt trội (Strengths):

- Tc bao phủ phân hoạch tng & giá trị biên (Domain & BVA):** AI sinh trong vài giây hàng chục ca kiểm thử với các giá trị biên kinh điển: chuỗi rỗng "", chuỗi 255 ký t, s âm, s 0, s th, và email thi u @.
- Kh năng cấu trúc hóa dữ liệu:** AI chuyển đổi từ v n b n t do thành các bng test case có trng thu tính mch l (TestID, Input, Expected Output).

1.2 Những mù & ảo giác của AI (Blind Spots & Hallucinations):

- Ảo giác về API Endpoints không tồn tại (Endpoint Hallucination):**
 - Thức test phát hiện trên FR-07 (Giỏ hàng): AI từng sinh ra các kịch bản kiểm thử gửi PUT /api/cart rồi copy nhúng số ID và DELETE /api/cart/:id để xóa item. Tuy nhiên, khi soi mã nguồn [server.js](#), SUT chỉ cài đặt duy nhất 2 endpoint GET và POST. AI đã suy diễn dựa trên logic thông thường mà không kiểm chứng mã nguồn thực tế.
- Mù hoàn toàn về ô nhiễm trạng thái chung kiểm thử (State Pollution / Domino Effect):**
 - Thức test phát hiện trên FR-02 (Khóa tài khoản): AI thiết kế các test case sai lầm khi dùng chung email test@eshop.com. Trong môi trường SUT có bug tăng +2 điểm lỗi sai, tài khoản test@eshop.com bị khóa vĩnh viễn (403), dẫn đến toàn bộ các test case xác thực phía sau trong cùng run đều bị gây (Flaky Run). Khi số con ngỗng phải can thiệp phân tách các email khác nhau (lockout_target@eshop.com, empty_pass_user@eshop.com).

2 KHÓA CỐNH 2: KỸ NĂNG SINH ASSERTIONS & SCHEMA VALIDATION

2.1 Những minh chứng test:

- Tạo JSON Schema chuẩn cú pháp Ajv/tv4:** AI sinh các khối schema { type: 'object', required: [...], properties: {...} } cho các phần hợp lệ thành công 200 OK rồi test nhanh và chuẩn xác.
- Tăng cường hóa kiểm tra thời gian phản hồi (Latency Checks):** Tăng thêm assertion `pm.expect(pm.response.responseTime).to.be.below(...)`.

2.2 Những mù & Hết chỗ:

- Ảo giác về HTTP Status Code chuẩn RESTful:**
 - AI mắc sai lầm cho rằng mọi request gửi email sai format hoặc body rỗng sẽ trả về 400 Bad Request. Nhưng thực tế SUT Express không validate đầu vào mà truy vấn SQLite trả về 401 unauthorized. Nếu chủ ý nguyên bản test của AI, test suite sẽ fail hàng loạt.
- Thiếu kiểm tra sâu cấu trúc Payload (Shallow Assertions):**
 - Mới với JWT Token, AI chỉ kiểm tra `pm.expect(jsonData.token).to.be.a('string')`. AI hoàn toàn bỏ qua việc giải mã Base64 payload để xác thực các claims nghi ngờ về quyền (payload.id, payload.role === 'admin', payload.iat). Khi số con ngỗng phải bổ sung `Buffer.from(parts[1], 'base64')` để kiểm tra chuyên sâu.

KHÓA CỐNH 3: PHÁT HIỆN LỖ HẦNG BẢO MẬT (OWASP API SECURITY TOP 10)

3.1 Tóm tắt:

- Thuộc lòng các vector tấn công Injection kinh điển: AI sinh ra tất cả các payload SQL Injection (' OR 1=1 --, admin' OR '1'='1) và XSS Script tags (<script>alert(1)</script>).

3.2 Lưu ý những điểm chí mấu chốt (Critical Security Blind Spots):

- Không phát hiện lỗ hổng rò rỉ dữ liệu nhúng cấp (SEC-01 - Broken Object Property Level Authorization):

- Trong POST /api/login, SUT trả về nguyên văn trả về password dạng plaintext trong object user. AI khi sinh test case Happy Path chỉ assert pm.expect(jsonData).to.have.property('user') và coi test case này là Pass, hoàn toàn không nhận ra đây là một lỗ hổng bảo mật cấp Critical.

- Không phát hiện lỗ hổng thiếu phân quyền Admin (SEC-03 - Broken Function Level Authorization):

- Trên các API CRUD sản phẩm (POST/PUT/DELETE /api/products), SUT hoàn toàn không gọi middleware authenticateToken. Bất kể ai không có token cũng xóa được sản phẩm. AI chỉ sinh test gọi kèm Authorization: Bearer <admin_token> nên test luôn Pass, bỏ sót lỗ hổng bảo mật nghiêm trọng nhất hệ thống.

3. Bảng Tổng Hợp Kiểm Toán AI Định Lượng (Quantitative AI Audit)

Phân hệ API	Tổng số TC AI sinh	Số TC Hợp lệ (VALID)	Số TC Không hợp lệ (INVALID)	Số TC Thiếu sót (INCOMPLETE)	Tỷ lệ AI cần Con người Can thiệp
FR-02 (Đăng nhập & Khóa TK)	38	31 (81.6%)	5 (13.2%)	2 (5.3%)	18.4%
FR-07 (Giới hàng)	38	30 (78.9%)	3 (7.9%)	5 (13.2%)	21.1%
FR-15 (Quản lý sản phẩm CRUD)	38	31 (81.6%)	5 (13.2%)	2 (5.3%)	18.4%
TỔNG CỘNG	114	92 (80.7%)	13 (11.4%)	9 (7.9%)	19.3%

Kết luận định lượng: Gần 1/5 số test cases do AI sinh ra (19.3%) chứa lỗi logic, gọi sai status code hoặc thiếu sót nghiêm trọng về mặt bảo mật và quản lý trạng thái CSDL.

4. Kết Luận: Vai Trò Không Thể Thay Thế Của KS QA Con Người

1. **AI là Bù Khuếch Đại Năng Suất (Productivity Multiplier), không phải Kéo Sụt Tốc Độ:** AI giúp giảm 70% thời gian viết boilerplate code, soạn JSON schema và tài liệu OpenAPI.
2. **Con người đóng vai trò Quality Gatekeeper & Security Auditor:** Chỉ có khả năng con người mới có năng lực:
3. Kiểm tra và phân tích trực tiếp mã nguồn thực tế (White-box Code Auditing) để tìm ra các lỗi ẩn giấu.
4. Thiết kế dữ liệu thử nghiệm thực tế, cách ly ảnh hưởng Domino làm sạch Test Suite.
5. Thêm vào các hệ thống bảo mật phi cấu trúc (Logical Access Control, Data Privacy).